Lesson 7 Worksheet: The Challenge of Turning — ANSWER KEY

Part 1: Heading from Coordinates

Current Position	Next Position	row_diff	col_diff	Heading (number and name)
(0, 0)	(1, 0)	1 - 0 = 1	0 - 0 = 0	2 (S)
(2, 3)	(1, 3)	1 - 2 = -1	3 - 3 = 0	0 (N)
(1, 1)	(1, 2)	0	1	1 (E)
(3, 2)	(3, 1)	0	-1	3 (W)
(0, 3)	(1, 3)	1	0	2 (S)
(2, 0)	(2, 1)	0	1	1 (E)
(3, 1)	(2, 1)	-1	0	0 (N)
(0, 2)	(0, 1)	0	-1	3 (W)

Part 2: The Right Turns Table

Current Needed	0 (N)	1 (E)	2 (S)	3 (W)
0 (N)	0	1	2	3
1 (E)	3	0	1	2
2 (S)	2	3	0	1
3 (W)	1	2	3	0

- How many cells have 0 right turns? 4 — Why? When the current heading equals the needed heading, `(needed - current) % 4 = 0`. These fall along the diagonal.
- What does 2 right turns mean physically? A 180-degree turn (two right turns = turning completely around).
- Do you notice a pattern in the table? Each row is the same sequence [0, 1, 2, 3] shifted. The values only depend on the difference between the headings, not the headings themselves.

Part 3: Determine the Number of Right Turns

Scenario	Current Heading	Needed Heading	(needed - current)	% 4	Right Turns
1	0 (N)	1 (E)	1 - 0 = 1	1 % 4 = 1	1
2	1 (E)	1 (E)	1 - 1 = 0	0 % 4 = 0	0
3	2 (S)	0 (N)	0 - 2 = -2	-2 % 4 = 2	2
4	3 (W)	2 (S)	2 - 3 = -1	-1 % 4 = 3	3
5	1 (E)	0 (N)	0 - 1 = -1	-1 % 4 = 3	3
6	0 (N)	3 (W)	3 - 0 = 3	3 % 4 = 3	3
7	2 (S)	3 (W)	3 - 2 = 1	1 % 4 = 1	1
8	3 (W)	1 (E)	1 - 3 = -2	-2 % 4 = 2	2

- Why does the `% 4` make negative numbers work correctly? In Python, the modulo operator always returns a non-negative result when the divisor is positive. So `-1 % 4 = 3` and `-2 % 4 = 2`, which correctly gives the number of clockwise right turns. A result of 3 means "3 right turns" (which is equivalent to 1 left turn, but we only use right turns).

Part 4: Trace a Path with Heading Updates

Path: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3)], Starting heading: 0 (N)

Step	From	To	Needed Heading	Current Heading	Right Turns	New Heading
1	(0, 0)	(1, 0)	2 (S)	0 (N)	(2-0)%4 = 2	2 (S)
2	(1, 0)	(2, 0)	2 (S)	2 (S)	(2-2)%4 = 0	2 (S)
3	(2, 0)	(2, 1)	1 (E)	2 (S)	(1-2)%4 = 3	1 (E)
4	(2, 1)	(2, 2)	1 (E)	1 (E)	(1-1)%4 = 0	1 (E)
5	(2, 2)	(2, 3)	1 (E)	1 (E)	(1-1)%4 = 0	1 (E)

- How many steps required turning? 2 (step 1 with 2 right turns and step 3 with 3 right turns)
- How many steps required zero turns? 3 (steps 2, 4, 5)
- At which step did the robot change from row to column movement? Step 3

Part 5: Trace a Second Path

Path: [(2, 3), (1, 3), (0, 3), (0, 2), (0, 1), (0, 0)], Starting heading: 2 (S)

Step	From	To	Needed Heading	Current Heading	Right Turns	New Heading
1	(2, 3)	(1, 3)	0 (N)	2 (S)	(0-2)%4 = 2	0 (N)
2	(1, 3)	(0, 3)	0 (N)	0 (N)	(0-0)%4 = 0	0 (N)
3	(0, 3)	(0, 2)	3 (W)	0 (N)	(3-0)%4 = 3	3 (W)
4	(0, 2)	(0, 1)	3 (W)	3 (W)	(3-3)%4 = 0	3 (W)
5	(0, 1)	(0, 0)	3 (W)	3 (W)	(3-3)%4 = 0	3 (W)

Final heading after completing path: 3 (W)

Part 6: Trace a Third Path

Path: [(1, 1), (1, 2), (1, 3), (2, 3), (3, 3)], Starting heading: 0 (N)

Step	From	To	Needed Heading	Current Heading	Right Turns	New Heading
1	(1, 1)	(1, 2)	1 (E)	0 (N)	(1-0)%4 = 1	1 (E)
2	(1, 2)	(1, 3)	1 (E)	1 (E)	(1-1)%4 = 0	1 (E)
3	(1, 3)	(2, 3)	2 (S)	1 (E)	(2-1)%4 = 1	2 (S)
4	(2, 3)	(3, 3)	2 (S)	2 (S)	(2-2)%4 = 0	2 (S)

- Final heading: 2 (S)
- Does the first step require turning? Yes — Why? The robot starts facing 0 (North) but needs heading 1 (East), so it must make 1 right turn: `(1-0)%4 = 1`.

Part 7: Robot Orientation Diagrams

- Scenario A: Facing 0 (N), need heading 1 (E) → Right turns needed: 1 — `(1-0)%4 = 1`
- Scenario B: Facing 1 (E), need heading 2 (S) → Right turns needed: 1 — `(2-1)%4 = 1`
- Scenario C: Facing 3 (W), need heading 1 (E) → Right turns needed: 2 — `(1-3)%4 = 2`
- Scenario D: Facing 2 (S), need heading 2 (S) → Right turns needed: 0 — `(2-2)%4 = 0`

Part 8: Heading and Turning Logic in Code

```
HEADING_NAMES = ["N", "E", "S", "W"]

def get_needed_heading(current_pos, next_pos):
    row_diff = next_pos[0] - current_pos[0]
```

```
    col_diff = next_pos[1] - current_pos[1]

    if row_diff == -1:
        return 0      # North
    elif col_diff == 1:
        return 1      # East
    elif row_diff == 1:
        return 2      # South
    elif col_diff == -1:
        return 3      # West

def count_right_turns(current, needed):
    return (needed - current) % 4
```

Test your understanding:

1. `count_right_turns(1, 0)` *returns 3 —* `(0-1)%4 = 3` *(three right turns, equivalent to turning 270 degrees clockwise)*
2. `count_right_turns(3, 3)` *returns 0 —* `(3-3)%4 = 0` *(no turn needed, already facing the right way)*
3. `count_right_turns(0, 2)` *returns 2 —* `(2-0)%4 = 2` *(two right turns, equivalent to a 180-degree turn)*

Reflection

Why is using numbers and a formula better than using strings and dictionaries? *Using numbers 0-3 lets us replace multiple dictionaries with a single formula:* `(needed - current) % 4`*. The formula works for all 16 combinations automatically – no need to build or maintain lookup tables. It's shorter to write, harder to get wrong, and reveals the underlying mathematical structure: headings are arranged in a circle, and the modulo operation naturally wraps around. This approach is also easier to extend – if you wanted to add diagonal headings, you would just use 8 directions instead of 4 with the same formula pattern.*